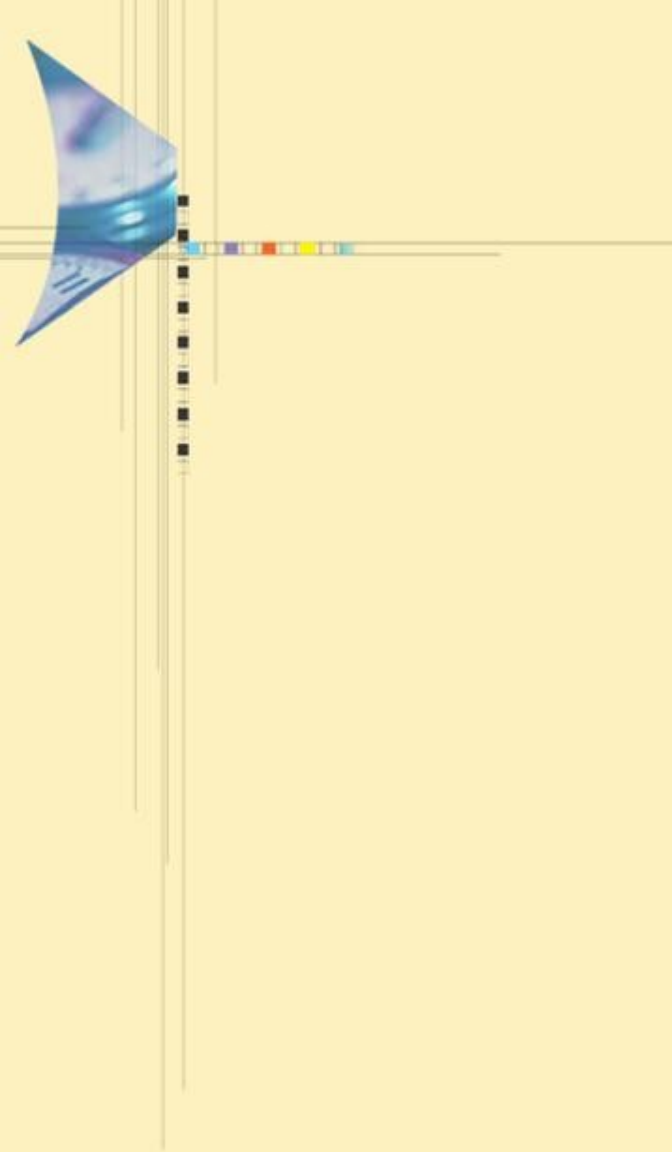




第 6 讲

图的遍历

——Traversal of Graphs



本章内容

- 6.1 引言
- 6.2 深度优先搜索
 - 6.2.1 无向图的情况
 - 6.2.2 有向图的情况
 - 6.2.3 深度优先搜索的时间复杂性
- 6.3 深度优先搜索的应用
 - 6.3.1 图的无回路性
 - 6.3.2 拓扑排序
 - 6.3.3 寻找图的关节点
 - 6.3.4 强联通分分支
- 6.4 广度优先搜索
- 6.5 广度优先搜索的应用
- 6.6 A* 搜索算法

6.1 引言

- 在一些诸如找出最短路径或最小生成树的图的算法中，用由它们相应的算法顺序访问顶点和边。
- 然而，在一些其他的算法中，访问顶点的顺序是不重要的，重要的是，不管输入的图如何，**用一种系统的顺序来访问顶点。**
- 在本章中，我们讨论图遍历的两种方法：深度优先搜索和广度优先搜索

6.2 深度优先搜索

- 深度优先搜索是一种强有力的遍历方法，它能帮助解决许多包含图的问题，它本质上是根树前序遍历的一个推广。
- 设 $G=(V,E)$ 是一个有向或无向图， G 的深度优先搜索运作如下：首先所有的顶点标上未访问，接着选择一个起始顶点，比如说 $v \in V$ ，并标上已访问。设 w 是邻接于 v 的任意一个顶点，我们把 w 标为已访问并且前进到另一个顶点，比如说称为 x ，它是邻接于 w 并且被标为未访问的。我们再把 x 标为已访问，并且前进到另一个邻接于 x 且标为未访问的顶点，这个选择邻接于当前顶点的一个未访问过的顶点的过程尽可能深地继续，直到我们找出一个顶点 y ，邻接于它的所有顶点都已被标上已访问。在这点处，我们返回到最后访问的顶点，比如说，如果还有邻接于它的任何未访问顶点，就访问它。继续这种方法，我们最终返回到起始顶点。
- 这种遍历方法已经给出了深度优先搜索的名称，因为它在朝前（深）的方向延伸搜索。这样一个遍历算法能用递归来写，如算法 DFS 所示或用一个堆栈写出。

6.2 深度优先搜索

这种遍历方法已经给出了深度优先搜索的名称，因为它在朝前（深）的方向延伸搜索。

- 这样一个遍历算法能用递归来写，如算法 DFS 所示或用一个堆栈写出。

算法 9.1 DFS

输入：有向或无向图 $G = (V, E)$ 。

输出：在相应的深度优先搜索树中对顶点的前序和后序。

1. $predfn \leftarrow 0; postdfn \leftarrow 0$
2. **for** 每个顶点 $v \in V$
3. 标记 v 未访问
4. **end for**
5. **for** 每个顶点 $v \in V$
6. **if** v 未访问 **then** $dfs(v)$
7. **end for**

过程 $dfs(v)$

1. 标记 v 已访问
2. $predfn \leftarrow predfn + 1$
3. **for** 每条边 $(v, w) \in E$
4. **if** w 标记为未访问 **then** $dfs(w)$
5. **end for**
6. $postdfn \leftarrow postdfn + 1$

6.2 深度优先搜索

- 算法从把所有顶点标上未访问开始，它还把两个计数器 $predfn$ 和 $postdfn$ 初始化为 0。然后，算法对 V 中每一个未访问顶点调用过程 dfs ，这是因为从起始点出发不是所有的顶点都可以到达的。从某个顶点 $v \in V$ 开始，过程 dfs 通过访问 v ，把 v 标成已访问，然后递归地访问它的邻接顶点来执行对 G 的搜索。当搜索完成时，如果从起始点开始所有的顶点都可到达，就构建起一棵生成树，称为深度优先搜索生成树，它的边是在前进方向上即当探索未访问顶点时被检查的那些边。
- 换句话说，设 (u, w) 是这样一条边， w 标有未访问，而整个过程假设是通过调用 $dfs(v)$ 来进行的，在这种情况下，这条边将成为深度优先搜索生成树的一部分。如果从起始点出发不是所有的顶点都是可到达的，那么搜索的结果产生一个生成树森林。在搜索完成后，每一个顶点标上了 $predfn$ 和 $postdfn$ 这两个标号，在深度优先搜索遍历产生的生成树（或森林）的顶点上加上了前序和后序号，它们给出了访问一个顶点的开始和结束的序。下面我们说边 (v, w) 已被探索的意思是指在调用 $dfs(v)$ 中，过程检查边 (u, w) 以测试 w 在这以前是否被访问过，根据图是有向或无向图的边将进行不同的分类。

6.2.1 深度优先搜索——无向图的情况

- 设 $G=(V,E)$ 是无向图，作为遍历的结果， G 的边分成以下两种类型。
 - 树边：深度优先搜索树中的边，如果在探索边 (v,w) 时， w 是首次被访问，则边 (v,w) 是树边；
 - 回边：所有其他的边。

6.2.1 深度优先搜索——无向图的情况

- 例 9.1 图 9.1(b) 阐明了在图 9.1(a) 所示的无向图上进行深度优先搜索
- 度优先
搜索树
先搜索
postdr
先搜索
点用

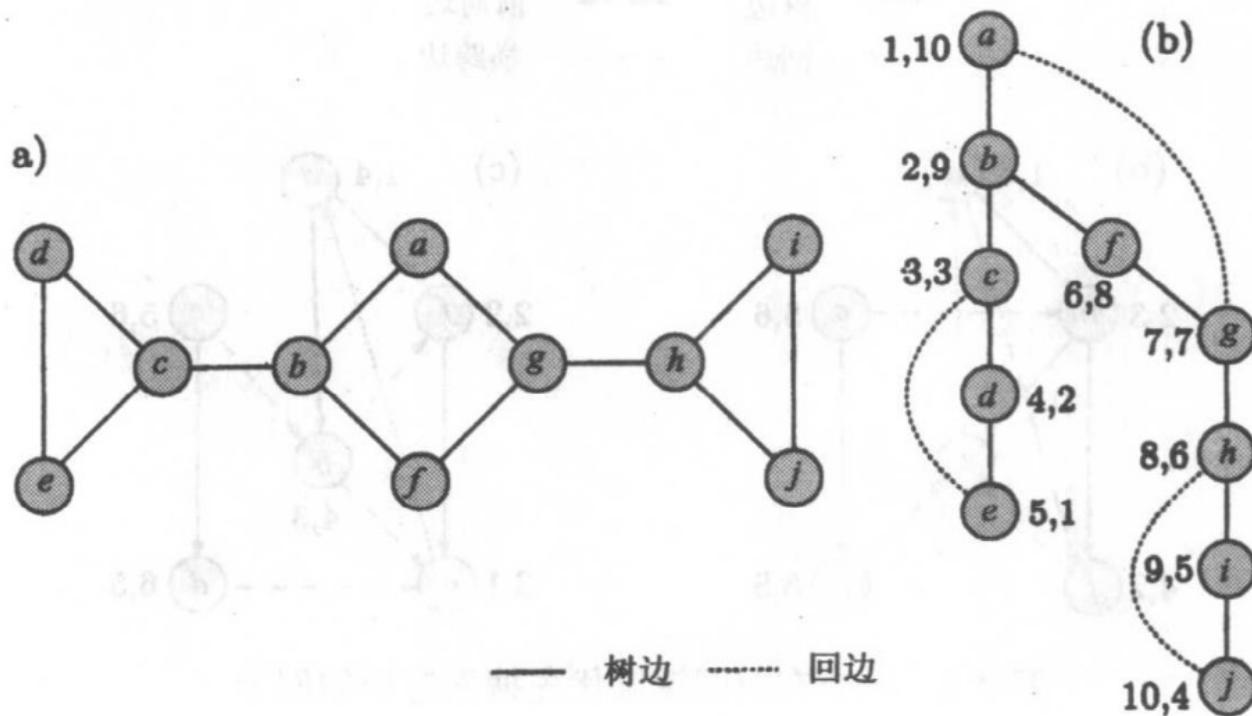


图 9.1 一个无向图深度优先搜索遍历的例子

6.2.2 深度优先搜索——有向图的情况

- 有向图中的深度优先搜索导致了一个或多个(有向)生成树,树的多少取决于起始点。如果 v 是起始点,深度优先搜索一棵树,它由从 v 出发所有可到达的顶点组成。如果那棵树没有包括所有的顶点,搜索从另一个未访问过的称为 w 的顶点再继续,构建一棵从 w 可到达的所有未访问顶点组成的树,这个过程继续下去直到所有的顶点都被访问过。
- 在有向图的深度优先搜索遍历中, G 的边分成 4 种类型。
 - 树边:深度优先搜索树中的边。如果在探索边 (u,w) 时, w 是第一次被访问,则 (v,w) 是树边;
 - 回边:在迄今为止构建的深度优先搜索树中, w 是 v 的祖先,并且当探索 (u,w) 时顶点 w 已被标上已访问,这样形式的边 (v,w) 是回边;
 - 前向边:在迄今为止构建的深度优先搜索树中, w 是 v 的后裔,并且当探索 (u,w) 时顶点 w 被标上已访问,这种形式的边 (v,w) 是前向边;
 - 横跨边:所有其他的边

6.2.2 深度优先搜索——有向图的情况

- 例 9.2
- 图 9.2(b) 阐明了在图 9.2(a) 行为从顶点 a 开始, 依次开始, 然后访问 d, 再访问一条回边, 因为在深度优先搜索中, 边 (a,f) 是 f 的祖先, 并且 (a,f) 不是互为祖先, 因此边 (f,e) 是棵不同的树, 它们是横跨边。不是 b, 这样的话, (a,b) 和搜索遍历的结果如图 9.2(c) 序。

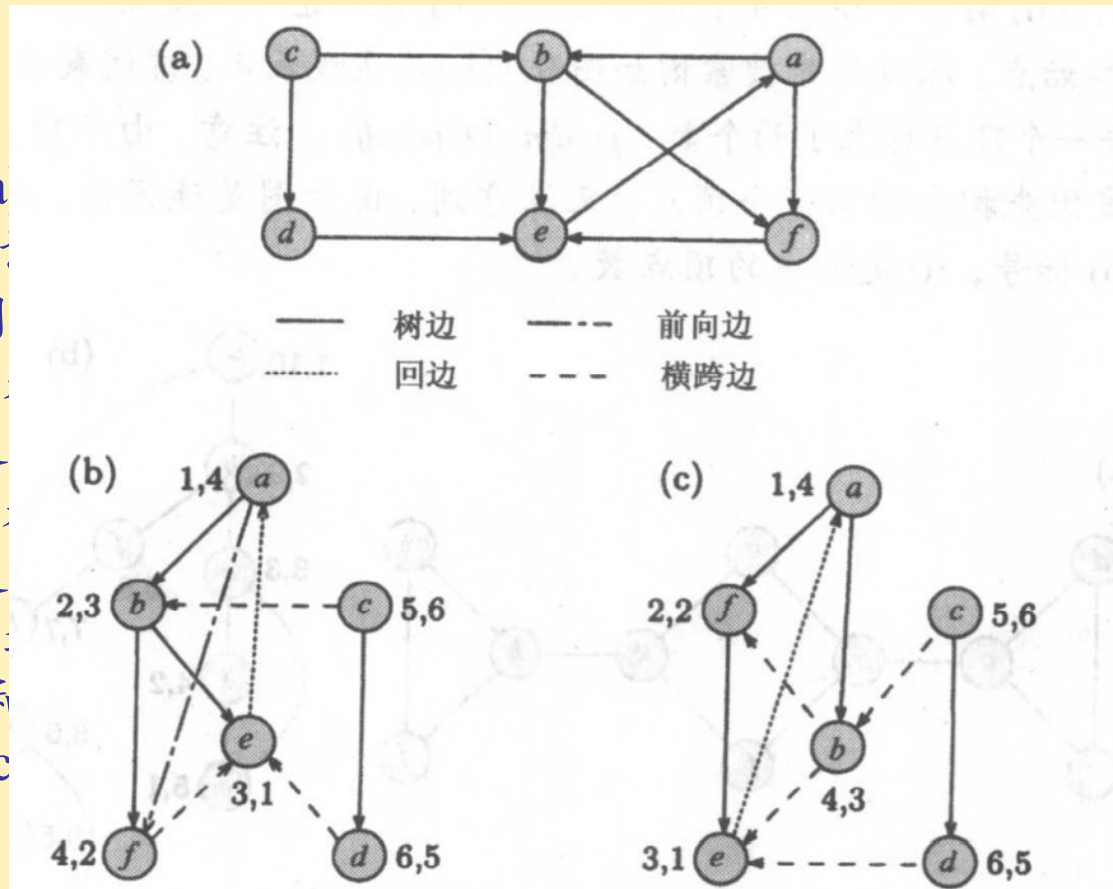


图 9.2 一个有向图深度优先搜索遍历的例子

6.2.3 深度优先搜索——时间复杂性

- 分析在具有 n 个顶点和 m 条边的图上用算法 DFS 时的时间复杂性。
- 由于一旦在顶点 v 上调用过程，就将对它标上已访问标记，因此在，上不会有更多的调用发生，于是过程调用的次数恰好是 n 次，如果把 for 循环排除在外，过程调用的耗费是 $\Theta(1)$ 。因此排除 for 循环后，过程调用的总耗费是 $\Theta(n)$ 。
- 算法的步骤 1 和步骤 3 分别耗费 $\Theta(1)$ 和 $\Theta(n)$ 时间，步骤 6 测试一个顶点是否已标记要耗费 $\Theta(n)$ ，现在还剩下找出过程中 for 循环的耗费。测试顶点 w 是否标记为未访问，这一步的执行次数等于顶点 v 的邻接点数。因此这一步执行的总次数，在有向图的情况下等于它的边数，在无向图的情况下是边数的两倍。于是，不论是有向图还是无向图，这一步的耗费是 $\Theta(m)$ ，这样算法运行的时间是 $\Theta(m+n)$ 。
- 如果图是连通的或有 $m \geq n$ ，则运行时间简单地为 $\Theta(m)$ 。但是必须强调的是，图假设是用邻接表表示的。

6.3 深度优先搜索的应用

- 深度优先搜索在图和几何算法中用得很普遍。这是一个强有力的工具，它有无数的应用，在本节中，我们列举它的一些重要应用。

6.3.1 图的无回路性

- 设 $G=(V,E)$ 是一个有几个顶点和 m 条边的有向或无向图。为了测试 G 是否至少有一个回路，我们对 G 用深度优先搜索，如果在搜索过程中探测到一条回边，那么是有回路的；否则 G 是无回路的。
- 注意 G 可能不连通，如果 G 是连通无向图，那么不需要对图进行遍历，因为 G 是无回路的当且仅当它是一棵树，则当且仅当 $m=n-1$ (见定理 3.1)。

6.3.2 拓扑排序

- 给出一个有向无回路图 (dag) $G=(V,E)$, 拓扑排序问题是找出图顶点的一个线性序, 使得如果 $(u,w) \in E$, 那么在这个序中 u 在 w 前出现。
- 例如, 在图 9.3(a) 所示的有向无回路图中, 顶点的一个可能的拓扑排序是 a,b,d,c,e,f,g 。我们将假设在这种有向无回路图中有一个人度是 0 的惟一的顶点 s , 如果不是的话, 我们可以简单地添一个新的点 s , 然后让 s 到所有人度为 0 的点加上边, 见图 9.3(b)。
- 接下来, 我们从顶点 s 起, 简单地对图 G 执行深度优先搜索。当遍历完成时, 计数器 $postdfn$ 定义了一个在有向无回路图中顶点的反拓扑序, 于是, 为了得到这个拓扑序, 可以在算法 DFS 中恰在计数器 $postdfn$ 刚好被增加后, 加上一个输出步, 把输出结果反转过来就是我们需

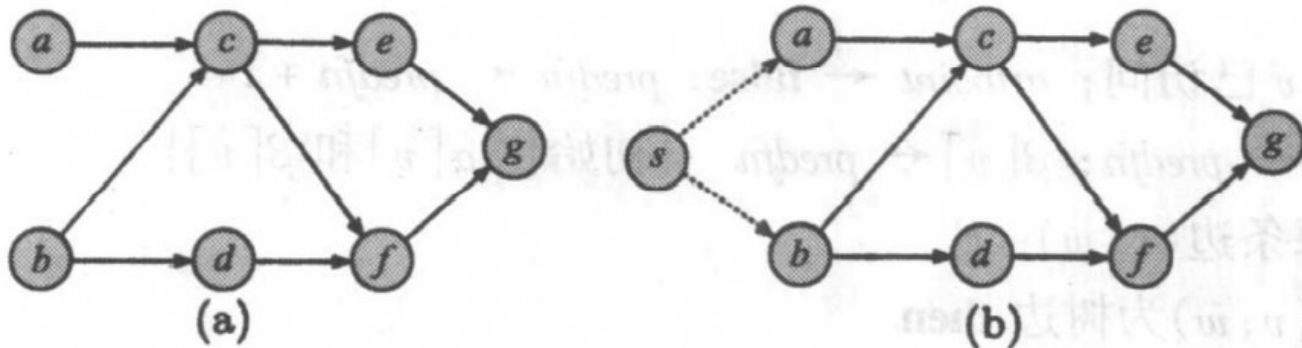


图 9.3 拓扑排序的图例

6.3.2 寻找图的关节点

- 在多于两个顶点的无向图 G 中, 存在一个顶点, 如果有不同于 v 的两个顶点 u 和 w , 在 u 和 w 间的任何路径都必定经过顶点 v , 则 v 称为关节点。这样, 如果 G 是连通的, 移去 v 和与它关联的边, 将产生 G 的不连通子图。一个图如果是连通的并且没有关节点则称为是双连通的。
- 为了找出关节点的集合, 在图 G 上执行一个深度优先搜索遍历, 在遍历的过程中, 对每个顶点 v 保持两个标号: $\alpha[v]$ 和 $\beta[v]$, $\alpha[v]$ 只是深度优先搜索算法中的 predfn , 每次调用深度优先搜索过程, 就加 1, $\beta[v]$ 初始化为 $\alpha[v]$, 但在后来的遍历过程中可以改变对于每一个访问的顶点, 令 $\beta[v]$ 是下面几个中的最小值。
 - $\alpha[v]$;
 - $\alpha[u]$, 对于每个顶点 u , (v,u) 是回边 ;
 - $\beta[w]$, 在深度优先搜索树中的每条边 (v,w) 。

6.3.3 寻找图的关节点

- 关节点确定如下：
- 根是一个关节点当且仅当在深度优先搜索树中，它有两个或更多的儿子；
- 根以外的顶点 v 是一个关节点当且仅当 v 有一个儿子 w , 使得 $\beta[w] \geq \alpha[v]$ 。

6.3.3 寻找图的关节点

算法 9.2 ARTICPOINTS

输入：连通无向图 $G = (V, E)$ 。

输出：包含 G 的所有可能关节点的数组 $A[1 \cdots \text{count}]$ 。

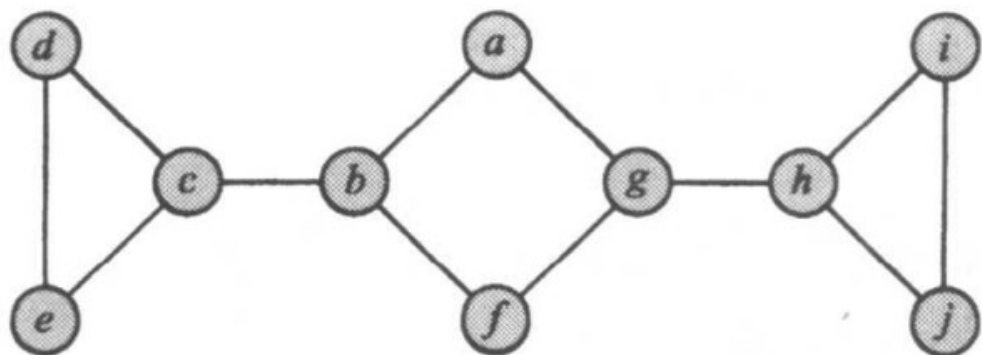
1. 设 s 为起始顶点
2. **for** 每个顶点 $v \in V$
3. 标记 v 未访问
4. **end for**
5. $\text{predfn} \leftarrow 0$; $\text{count} \leftarrow 0$; $\text{rootdegree} \leftarrow 0$
6. $\text{dfs}(s)$

过程 $\text{dfs}(v)$

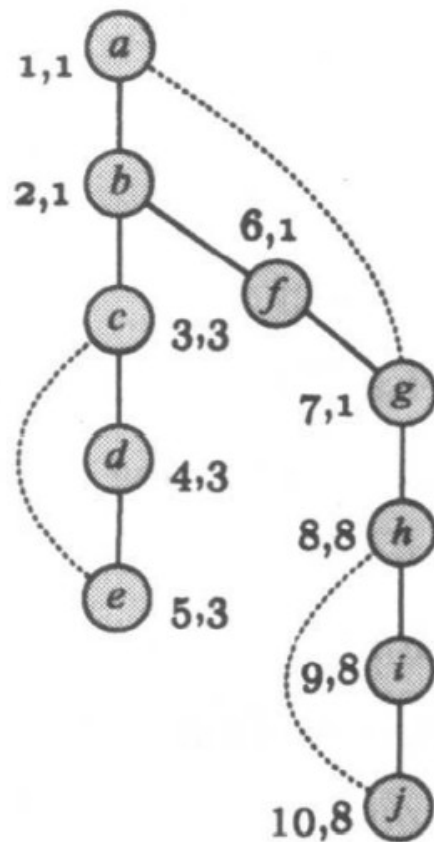
1. 标记 v 已访问; $\text{artpoint} \leftarrow \text{false}$; $\text{predfn} \leftarrow \text{predfn} + 1$
2. $\alpha[v] \leftarrow \text{predfn}$; $\beta[v] \leftarrow \text{predfn}$ {初始化 $\alpha[v]$ 和 $\beta[v]$ }
3. **for** 每条边 $(v, w) \in E$
4. **if** (v, w) 为树边 **then**
5. $\text{dfs}(w)$
6. **if** $v = s$ **then**
7. $\text{rootdegree} \leftarrow \text{rootdegree} + 1$
8. **if** $\text{rootdegree} = 2$ **then** $\text{artpoint} \leftarrow \text{true}$

9. **else**
10. $\beta[v] \leftarrow \min\{\beta[v], \beta[w]\}$
11. **if** $\beta[w] \geq \alpha[v]$ **then** $\text{artpoint} \leftarrow \text{true}$
12. **end if**
13. **else if** (v, w) 是回边 **then** $\beta[v] \leftarrow \min\{\beta[v], \alpha[w]\}$
14. **else do nothing** { w 是 v 的父亲}
15. **end if**
16. **end for**
17. **if** artpoint **then**
18. $\text{count} \leftarrow \text{count} + 1$
19. $A[\text{count}] \leftarrow v$
20. **end if**

6.3.3 寻找图的关节点

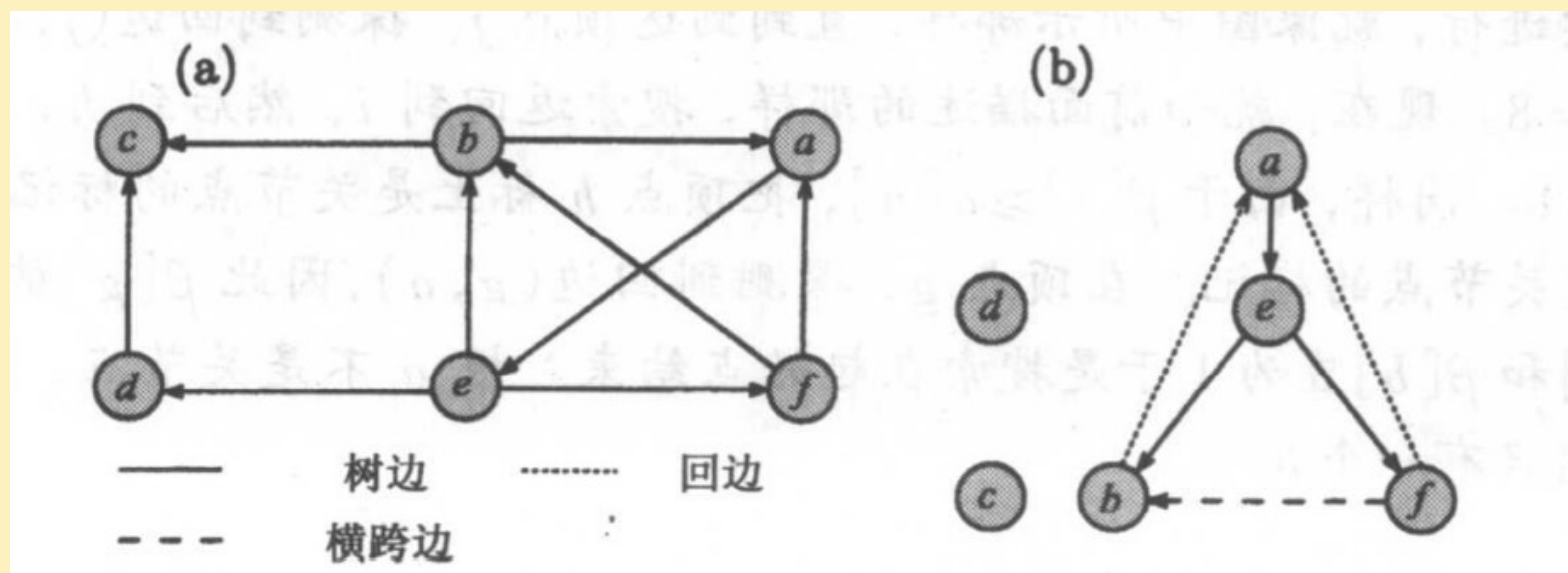


—— 树边 回边



6.3.4 强连通分支

- 给出一个有向图 $G=(V,E)$, G 中的强连通分支是它顶点的极大集, 在该集中, 每一对顶点间都存在一条路径。
- 算法 STRONGCONNECTCOMP 用深度优先搜索来确定在有向图中的所有的强连通分支。



6.3.4 强连通分支

- STRONCCONNECTCOMP 算法
- 输入：有向图 $G=(V,E)$ 。
- 输出：G 中的强连通分支

1. 在图 G 上执行深度优先搜索，对每一个顶点赋给相应的 $postdfn$ 值。
2. 颠倒图 G 中边的方向，构成一个新的图 G' 。
3. 从具有最大 $postdfn$ 数值的顶点开始，在 G' 上执行深度优先搜索，如果深度优先搜索不能到达所有的顶点，则在余下的顶点中找一个 $postdfn$ 数值最大的顶点开始下一次深度优先搜索。
4. 在最终得到的森林中的每一棵树对应一个强连通分支。

6.4 广度优先搜索

- 与深度优先搜索不同，在广度优先搜索中，当访问了一个顶点后，接下来访问邻接于 v 的所有顶点，产生出来的树称为广度优先搜索树。这种遍历的方法可以通过用一个队列存储没有检查过的顶点来实现。
- 广度优先的算法 BFS 能够用到有向和无向图。初始的时候，所有的顶点都标上未访问，计数器 bfn 初始为 0，它表示顶点从队列中移出的序。在无向图的情况下，一条边或者是树边或者是横跨边；如果图是有向的，那么边可以是树边、回边或横跨边，不存在前向边。

6.4 广度优先搜索

算法 9.4 BFS

输入：有向或无向图 $G = (V, E)$ 。

输出：广度优先搜索次序中顶点的编号。

1. $bf_n \leftarrow 0$
2. **for** 每个顶点 $v \in V$
3. 标记 v 未访问
4. **end for**
5. **for** 每个顶点 $v \in V$
6. **if** v 标记为未访问 **then** $bfs(v)$
7. **end for**

过程 $bfs(v)$

1. $Q \leftarrow \{v\}$
2. 标记 v 已访问
3. **while** $Q \neq \{\}$
4. $v \leftarrow Pop(Q)$
5. $bf_n \leftarrow bf_n + 1$
6. **for** 每条边 $(v, w) \in E$
7. **if** w 标记为未访问 **then**
8. $Push(w, Q)$
9. 标记 w 已访问
10. **end if**
11. **end for**
12. **end while**

6.4 广度优先搜索

- 图 9.7 说明了当把广度优先搜索遍历用到图 9.1(a) 所示的图上，并以顶点 a 为起始点时的运作情况。在弹出顶点 a 后，就把顶点 b 和 g 压入队列并且标上已访问，接着把顶点 b 从队列中移出，并且把它的还没有被访问过的邻接点 c 和 f 压入队列，标上已访问。这个把顶点压入队列以后又把它们从队列中移出来的过程一直继续，直到顶点 i 最终从队列中移出。这时，队列变空，广度优先搜索遍历也完成了。在图中，每一个顶点标上了它的 bfn 数，这就是该顶点从队列中移出的序。

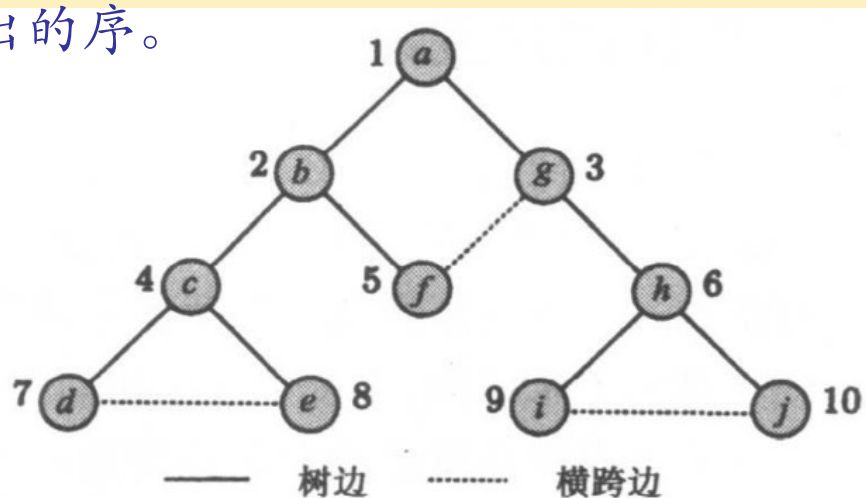


图 9.7 一个无向图广度优先搜索遍历的例子

6.4 广度优先搜索——时间复杂度分析

- 广度优先搜索应用在具有几个顶点 n 条边 m 的图 (包括有向图和无向图) 上时, 它的时间复杂性是和深度优先搜索一样的, 为 $\Theta(n+m)$, 如果图是连通的或者 $m \geq n$, 那么时间复杂性可以简化为 $\Theta(m)$ 。

6.5 广度优先搜索的应用

- 广度优先搜索的应用在图和网络算法中是很重要的。
- 设 $G=(V,E)$ 是连通无向图， s 是 V 中的一个顶点，当把算法 BFS 用到 G 上并以 s 作为起始点时，产生的广度优先搜索树中从 s 点到其他任意顶点的路径有最少的边数。这样，假如我们要找出从 s 到其他每一顶点的距离，这里从 s 到一个顶点 v 的距离定义为：从 s 到 v 的任意路径的最少边数。于是上面的问题可以很容易做到，我们只要在每个顶点压入队列前，标上它的距离就可以了。这样，起始点将标上 0，它的邻接点是 1，依次类推。很明显，每个顶点的标号是它到起始点的最短距离。例如在图 9.7 中，顶点 a 将被标上 0，顶点 b 和 g 标上 1，顶点 c, f 和 h 标上 2，最后，顶点 d, e, i 和 j 标上 3。

6.6 A* 搜索算法

- A* 算法是启发式搜索算法，是根据 Dijkstra 算法改进而来。
- 定义：是一种在图形平面上，对于有多个节点的路径求出最低通过成本的算法。它属于图遍历和最佳优先搜索算法，亦是 BFS 的改进。
- 在 BFS 搜索算法中，如果能在搜索的每一步都利用估价函数 $f(n)=g(n)+h(n)$ 对 Open 表（队列）中的节点进行排序，则该搜索算法为 A 算法。由于估价函数中带有问题自身的启发性信息，因此，A 算法又称为启发式搜索算法。
- 对启发式搜索算法，又可根据搜索过程中选择扩展节点的范围，将其分为全局择优搜索算法和局部择优搜索算法。
- A* 搜索算法是一种用于路径搜索和图遍历的效果很好、主流的技术之一。

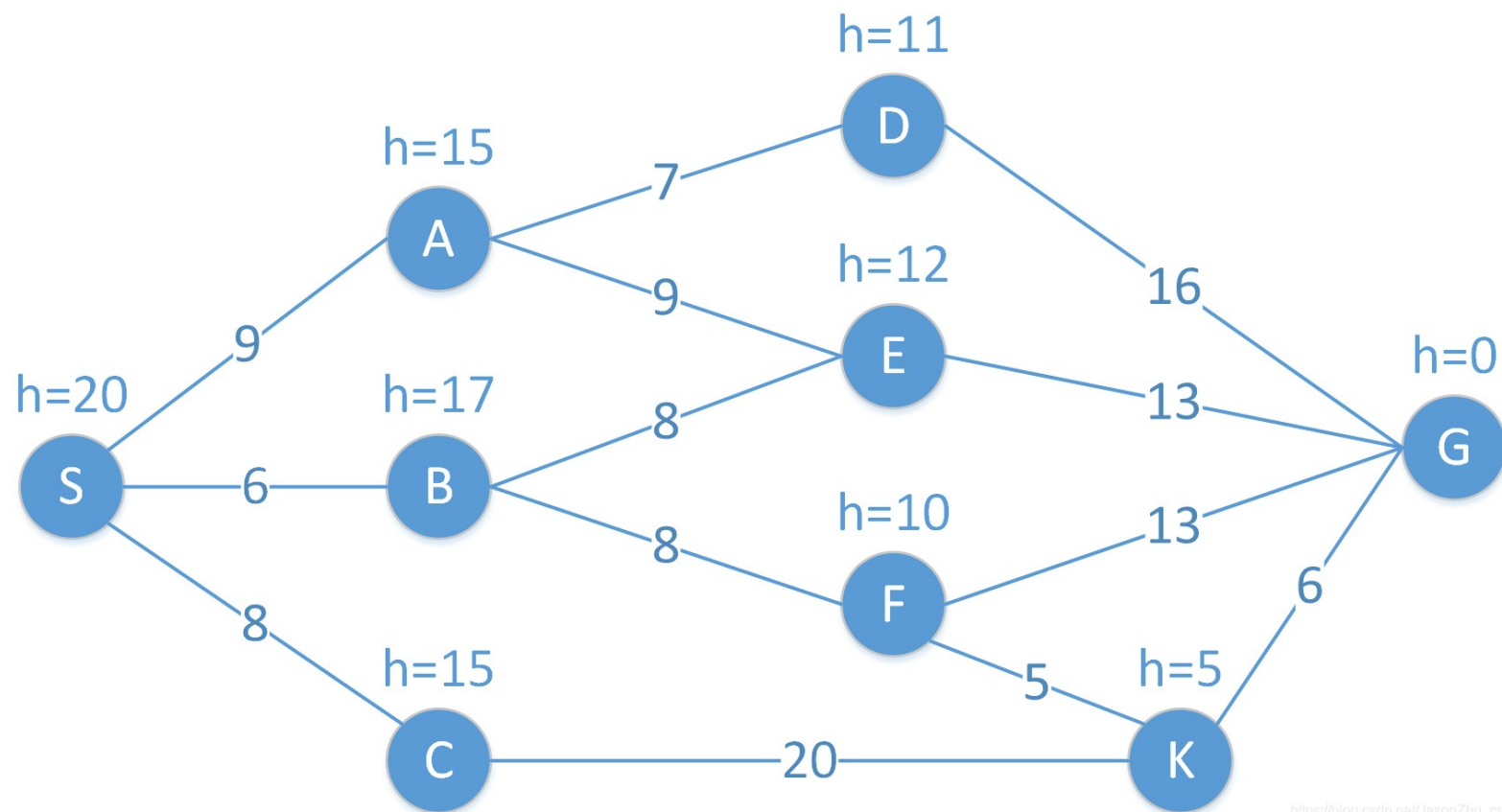
6.6 A* 搜索算法

- A* 算法是启发式搜索算法，是根据 Dijkstra 算法改进而来。
- **定义：** A* 算法是一种在图形平面上，对于有多个节点的路径求出最低通过成本的算法。它属于图遍历和最佳优先搜索算法，亦是 BFS 的改进。
- 在 BFS 搜索算法中，如果能在搜索的每一步都利用估价函数 $f(n)=g(n)+h(n)$ 对 Open 表 (队列) 中的节点进行排序，则该搜索算法为 A 算法。由于估价函数中带有问题自身的启发性信息，因此，A 算法又称为启发式搜索算法。
- 对启发式搜索算法，又可根据搜索过程中选择扩展节点的范围，将其分为全局择优搜索算法和局部择优搜索算法。
- A* 搜索算法是一种用于路径搜索和图遍历的效果很好、主流的技术之一。

6.6 A* 搜索算法

- 如何更好的理解 A* 算法?
- 如下图所示, S 为起始 (start) 节点, G 为目标 (goal) 节点。
- (1) 节点之间连线是两点的路径长度, 如 A 到 E 的路径长度 $c(A,E) = 9$ 。
- (2) 节点旁的 h 值是当前节点到达目标节点 (G) 的预估值, 如 $h(A)=15$, 表示从当前点 A 到达目标点 G 的估计路径长度为 15, 此处 $h(x)$ 即为启发函数。
- (3) 从起点 S 到达当前节点 x 的路径长度表示为 $g(x)$ 。
- (4) 从起点 S 到达目标 G 并经过点 x 的估计距离长度表示为 $f(x) = g(x) + h(x)$, 该公式是 A* 算法的核心公式。
- (5) A* 算法通过不断的选择估计距离 f 最小的节点, 逐渐构建最短路径。

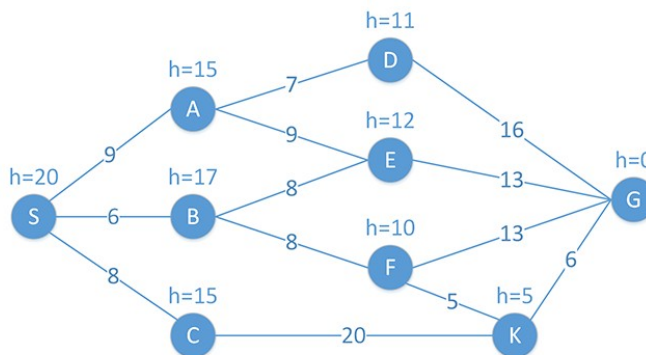
6.6 A* 搜索算法



https://blog.csdn.net/JasonZhu_csdn

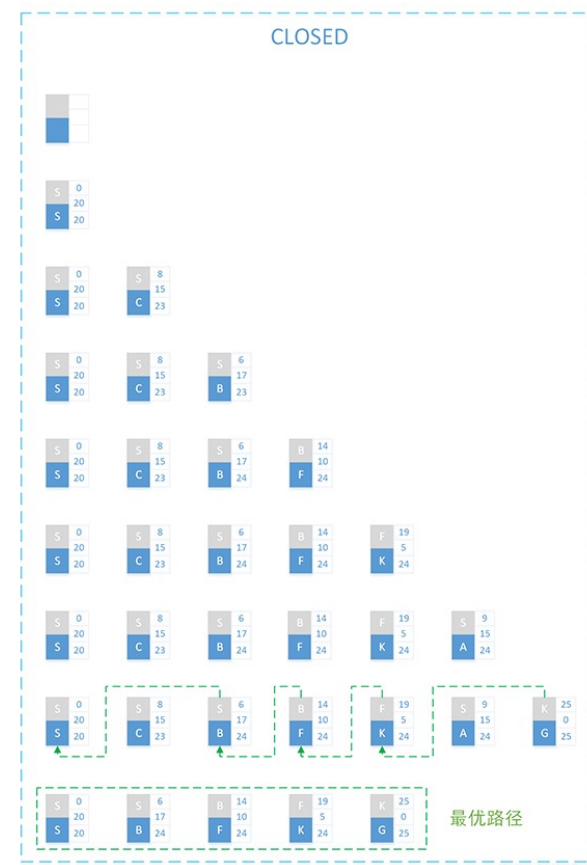
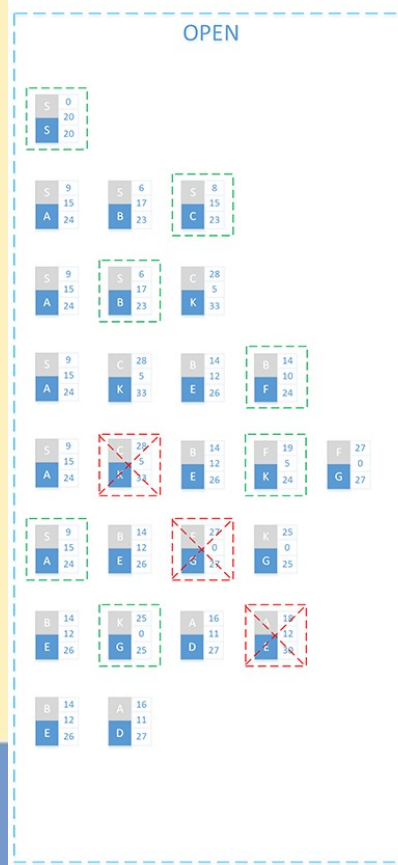
6.6 A* 搜索算法

- 搜索过程：
- 组合块中：
 - (a) 灰色为前序节点
 - (b) 蓝色当前节点 x
 - (c) g：起点 S 到当前节点 x 的路径距离。
 - (d) h：当前节点 x 到终点 G 的估计距离
 - (e) f：起点 S 途径 x 到达终点 G 的估计距离，即 $f = g + h$
 - (f) 绿色框为当前 OPEN 集合中的最优节点
 - (g) 红色框表示当前 OPEN 集合中需要被删除的节点
- 在 OPEN、CLOSED 中每一行表示一次完整迭代完成时两集合中的节点集合。
- 最后的最优路径是：S->B->F->k->G
- 注：当两个节点 f 相同时，h 小的更优



S	0
S	20

前序	g
当前	h
	f



6.6 A* 搜索算法

- 创建两个集合 OPEN 集，CLOSED 集，算法核心是从 OPEN 集中选择最优（ f 值小最优，或 f 相同时， h 小的更优）的节点到 CLOSED 集中，然后将其后继节点放入 OPEN 集中，然后重复操作选取最优节点，直到到达目标，或者 OPEN 为空为止。最后再 CLOSED 集中根据目标 G 所包含的前序节点逆序查找最后到达起点 S ，这个链路的逆序即最优路径，

6.6 A* 搜索算法

- 在全局择优搜索中，每当需要扩展节点时，总是从 Open 表的所有节点中选择一个估价函数值最小的节点进行扩展。其搜索过程可能描述如下：
 - (1) 把初始节点 S_0 放入 Open 表中， $f(S_0)=g(S_0)+h(S_0)$ ；
 - (2) 如果 Open 表为空，则问题无解，失败退出；
 - (3) 把 Open 表的第一个节点取出放入 Closed 表，并记该节点为 n ；
 - (4) 考察节点 n 是否为目标节点。若是，则找到了问题的解，成功退出；
 - (5) 若节点 n 不可扩展，则转到第 (2) 步；
 - (6) 扩展节点生成子节点 n_i ($i=1,2,3\dots$)，计算每一个子节点的估价值 $f(n_i)(i=1,2,3\dots)$ ，并为每一个子节点设置指向父节点的指针，然后将这些子节点放入 Open 表中；
 - (7) 根据各节点的估价函数值，对 Open 表中的全部节点按从小到大的顺序重新进行排序；
 - (8) 转第 (2) 步。

6.6 A* 搜索算法

- 由于上述算法的第 (7) 步要对 Open 表中的全部节点按其估价函数值从小到大重新进行排序，这样在算法第 (3) 步取出的节点就一定是 Open 表的所有节点中估价函数值最小的一个节点。因此，它是一种全局择优的搜索方式。
- 对上述算法进一步分析还可以发现：如果取估价函数 $f(n)=q(n)$ ，则它将退化为代价树的广度优先搜索；如果取估价函数 $f(n)=d(n)$ ，则它将退化为广度优先搜索。可见，广度优先搜索和代价树的广度优先搜索是全局择优搜索的两个特例 ($g(n)$ 是从起点到 n 的代价， $d(n)$ 是从起点到 n 经过的步数 (n 在 BFS 搜索树中的层次数))